Лабораторная работа №7

Принципы SOLID и рефакторинг архитектуры Python-приложения

1. Цель работы

Освоить принципы SOLID и рефакторинг через разделение ответственности, интерфейсы Protocol и внедрение зависимостей при разработке сервиса доставок.

2. Постановка индивидуального задания

Вариант 12 — «Курьерская служба». Выполнить рефакторинг приложения в соответствии с принципами SOLID.

Программа должна:

– Создать доставку, назначать курьера, менять статус и рассчитывать стоимость.

– Показать проблемы исходной архитектуры и провести рефакторинг с учётом пяти принципов SOLID.

– Реализовать заменяемые способы уведомления клиента и тесты.

3. Краткие теоретические сведения

SOLID — совокупность принципов построения поддерживаемых объектных систем: SRP (одна ответственность), OCP (расширение без изменения существующего кода), LSP (корректная подстановка), ISP (узкие интерфейсы), DIP (зависимость от абстракций).

В Python интерфейсы удобно описывать через typing.Protocol. Классы-потребители получают необходимые объекты извне с помощью Dependency Injection, а не создают жёстко привязанные реализации внутри себя.

Рефакторинг изменяет внутреннюю структуру без намеренного изменения внешнего поведения. Тесты подтверждают сохранение операций добавления, назначения курьера, смены статуса, расчёта цены и заменяемости уведомителей.

4. Исходный код программы

Файл before.py

```
# До SOLID: хранение, расчёт и уведомление смешаны в одном классе.
class BigInterface:
    def save(self):
        raise NotImplementedError

    def send_sms(self):
        raise NotImplementedError

    def print_report(self):
        raise NotImplementedError


class ConsoleNotifier(BigInterface):
    def send_sms(self):
        print("SMS отправлено (учебный вывод)")
    # save и print_report этому уведомителю не нужны: нарушение ISP.


class BadCourier:
    def deliver(self):
        return "Доставка выполнена"


class OfficeWorker(BadCourier):
    def deliver(self):
        raise ValueError("Офисный сотрудник не доставляет посылки")
    # Подтип не заменяет курьера: нарушение LSP.


class BadDeliveryService:
    def __init__(self):
        self.items = {}
        self.notifier = ConsoleNotifier()  # Жёсткая зависимость: нарушение DIP.

    def create(self, id, address, weight, distance):
        self.items[id] = dict(address=address, weight=weight, distance=distance,
                              courier="", status="создана")

    def assign(self, id, courier):
        self.items[id]["courier"] = courier

    def change_status(self, id, status, notification):
        item = self.items[id]
        transitions = {"создана": ["принята", "отменена"],
                       "принята": ["в пути", "отменена"],
                       "в пути": ["доставлена", "отменена"]}
        if status not in transitions.get(item["status"], []):
            raise ValueError("Недопустимый переход")
        if status == "в пути" and not item["courier"]:
            raise ValueError("Нет курьера")
        item["status"] = status
        if notification == "email":
            print("Email:", status)
```

```
        elif notification == "sms":
            print("SMS:", status)

    def cost(self, id):
        item = self.items[id]
        return 100 + item["distance"] * 20 + item["weight"] * 10
```

Файл models.py

```
from dataclasses import dataclass
from enum import Enum


class Status(str, Enum):
    CREATED = "создана"
    ACCEPTED = "принята"
    MOVING = "в пути"
    DELIVERED = "доставлена"
    CANCELLED = "отменена"


@dataclass
class Delivery:
    id: int
    address: str
    weight: float
    distance: float
    courier: str = ""
    status: Status = Status.CREATED


@dataclass
class DocumentDelivery(Delivery):
    document_count: int = 1


@dataclass
class ParcelDelivery(Delivery):
    fragile: bool = False
```

Файл repository.py

```
from typing import Protocol
from models import Delivery


class Reader(Protocol):
    def get(self, id: int) -> Delivery: ...
    def get_all(self) -> list[Delivery]: ...


class Writer(Protocol):
    def add(self, delivery: Delivery) -> None: ...
    def delete(self, id: int) -> None: ...
```

```
class Repository(Reader, Writer, Protocol):
    pass


class MemoryRepository:
    def __init__(self) -> None:
        self.items: dict[int, Delivery] = {}

    def add(self, delivery: Delivery) -> None:
        if delivery.id in self.items:
            raise ValueError("id уже существует")
        self.items[delivery.id] = delivery

    def get(self, id: int) -> Delivery:
        if id not in self.items:
            raise ValueError("Доставка не найдена")
        return self.items[id]

    def get_all(self) -> list[Delivery]:
        return list(self.items.values())

    def delete(self, id: int) -> None:
        self.get(id)
        del self.items[id]
```

Файл services.py

```
from typing import Protocol
from models import Delivery, Status
from repository import Repository


class Notifier(Protocol):
    def send(self, message: str) -> None: ...


class EmailNotifier:
    def send(self, message: str) -> None:
        print("Email (учебный вывод):", message)


class SmsNotifier:
    def send(self, message: str) -> None:
        print("SMS (учебный вывод):", message)


class DeliveryService:
    def __init__(self, repository: Repository, notifier: Notifier) -> None:
        self.repository = repository
        self.notifier = notifier

    def create(self, delivery: Delivery) -> None:
      if delivery.id <= 0 or not delivery.address or delivery.weight <= 0 or delivery.distance < 0:
            raise ValueError("Некорректная доставка")
        self.repository.add(delivery)
```

```
    def assign(self, id: int, courier: str) -> None:
        if not courier:
            raise ValueError("Укажите курьера")
        self.repository.get(id).courier = courier

    def change_status(self, id: int, status: str) -> None:
        delivery = self.repository.get(id)
        transitions = {"создана": ["принята", "отменена"],
                       "принята": ["в пути", "отменена"],
                       "в пути": ["доставлена", "отменена"]}
        if status not in transitions.get(delivery.status, []):
            raise ValueError("Недопустимый переход статуса")
        if status == "в пути" and not delivery.courier:
            raise ValueError("Сначала назначьте курьера")
        delivery.status = Status(status)
        self.notifier.send(f"Доставка {id}: {status}")

    def cost(self, id: int) -> float:
        delivery = self.repository.get(id)
        return 100 + delivery.distance * 20 + delivery.weight * 10
```

Файл main.py

```
from models import DocumentDelivery, ParcelDelivery
from repository import MemoryRepository
from services import DeliveryService, EmailNotifier, SmsNotifier


def main() -> None:
    for notifier in (EmailNotifier(), SmsNotifier()):
        repository = MemoryRepository()
        service = DeliveryService(repository, notifier)
        service.create(DocumentDelivery(1, "ул. Пушкина, 10", 2, 5, document_count=3))
        service.create(ParcelDelivery(2, "ул. Ленина, 5", 4, 8, fragile=True))
        service.assign(1, "Максим")
        service.change_status(1, "принята")
        service.change_status(1, "в пути")
        print("Стоимость:", service.cost(1))
        print(repository.get_all())


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 7.1 приведён фрагмент контрольного запуска рефакторинга сервиса доставки с заменой способов уведомления.

```
python main.py
Email (учебный вывод): Доставка 1: принята
Email (учебный вывод): Доставка 1: в пути
Стоимость: 220
SMS (учебный вывод): Доставка 1: принята
SMS (учебный вывод): Доставка 1: в пути
Стоимость: 220
```

Рисунок 7.1 – Результат контрольного запуска программы

Автоматическое тестирование выполнено с помощью pytest. Все 6 тестов прошли успешно (рисунок 7.2).

```
pytest -q
......
[100%]
6 passed in 0.03s
```

Рисунок 7.2 – Результат автоматического тестирования

6. Вывод

Проведён рефакторинг сервиса доставок по принципам SOLID: разделены модели, хранение, бизнес-логика и уведомления. Проверены оба варианта уведомлений и автоматические тесты (6 успешных).

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	